

UNIVERSITÀ POLITECNICA DELLE MARCHE

INGEGNERIA INFORMATICA E DELL'AUTOMAZIONE

Controllo di un motore brushless con feedback dell'encoder su STM32H745ZI-Q



Corso di
LABORATORIO DI AUTOMAZIONE

Anno accademico 2024-2025

Studente:

Toyem Tezem Ryan Parfait

Professore:

Andrea Bonci

Dottorandi:

Serafini Andrea

Pellicani Ilaria

Di Biase Alessandro



Dipartimento di Ingegneria dell'Informazione

Indice

Introduzione	1
1 Hardware	3
1.1 Il microcontrollore STM32H745ZI-Q	3
1.2 Motore DC modello 28PA51G	4
1.3 Alimentatore DC da laboratorio	6
1.4 Encoder adapter e DC motor adapter	7
1.5 Schema dei collegamenti	7
2 Software	10
2.1 Diagramma di flusso	10
2.2 Gestione singoli componenti	11
2.2.1 Motore	11
2.2.2 Encoder	12
2.2.3 Protocollo di comunicazione USART : USART3	14
2.3 Funzionamento complessivo	16
2.3.1 Spiegazione del calcolo effettivo	17
3 Test e risultati	19
4 Conclusioni e sviluppi futuri	21

Introduzione

In questo progetto è stato richiesto di avviare e controllare un motore brushless, il quale è dotato di un encoder integrato per la misura della velocità.

L'obiettivo del lavoro riguarda lo sviluppo di un programma sul microcontrollore al fine di far funzionare il sistema autonomamente, fornendo l'alimentazione necessaria e visualizzando in tempo reale tramite il software PuTTY le informazioni rilevate dall'encoder.

Il Capitolo 1 è dedicato alla presentazione dei componenti fisici del sistema e alle loro interconnessioni. Nel Capitolo 2, verrà illustrato il codice, partendo dall'IDE utilizzato fino all'implementazione dei diversi componenti hardware. Successivamente, il Capitolo 3 descriverà l'approccio e le metodologie adottate per testare il sistema, con la relativa illustrazione dei risultati ottenuti. Infine, il Capitolo 4 conclude il lavoro, proponendo riflessioni finali e possibili sviluppi futuri.

1 Hardware

Il sistema hardware è composto da tre elementi fondamentali, tra cui abbiamo effettuato le relative connessioni. Per semplificare i collegamenti, abbiamo utilizzato due schede adattatrici: un encoder adapter e un DC motor adapter.

In modo esaustivo, il sistema è composto da :

- Un microcontrollore STM32H745ZI-Q
- Un motore con encoder integrato, modello DC Motor 28PA51G
- Un alimentatore da laboratorio per l'alimentazione del sistema
- Due adattatori: un encoder adapter specifico per il motore e un DC motor adapter, modello Cytron MD10C

1.1 Il microcontrollore STM32H745ZI-Q

Nel corso del nostro lavoro, abbiamo utilizzato un microcontrollore dell'azienda STMicroelectronics, appartenente alla serie STM32 NUCLEO, e in particolare il modello STM32 NUCLEO-144¹ STM32H745ZI-Q².

Questo componente rappresenta il cuore del nostro lavoro. Le prime ore in laboratorio sono state dedicate alla sua familiarizzazione. Per interagire con il dispositivo, abbiamo utilizzato un cavo USB Type-A to Micro-B per instaurare la connessione tra la scheda di sviluppo e il PC.

Di seguito è riportata un'immagine illustrativa del microcontrollore, insieme ai pin presenti su di esso.

¹NOTA: La specifica 144 definisce il numero di pin sul microcontrollore, nella serie dei STM32 NUCLEO ne abbiamo tre tipi : 32, 64, 144.

²STM, <https://www.st.com/en/evaluation-tools/nucleo-h745zi-q.html>

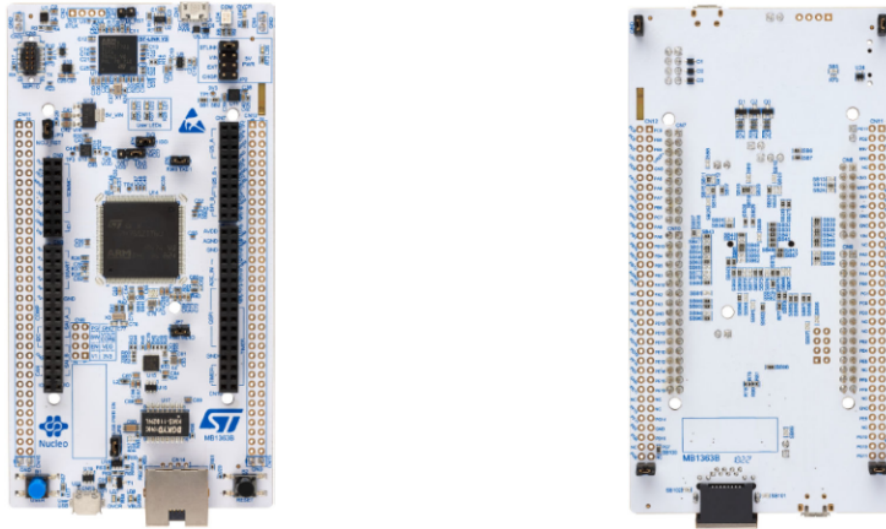


Figure 1.1: Foto illustrativa del microcontrollore

CN8				CN7				
NC	NC	1 2	D43	PC6	D16	1 2	D15	PB8
IOREF	IOREF	3 4	D44	PB15	D17	3 4	D14	PB9
NRST	RESET	5 6	D45	PB13	D18	5 6	AVDD	VREFP
3V3	+3V3	7 8	D46	PB12	D19	7 8	GND	GND
5V	+5V	9 10	D47	PA15	D20	9 10	D13	PA5
GND	GND	11 12	D48	PC7	D21	11 12	D12	PA6
GND	GND	13 14	D49	PB5	D22	13 14	D11	PB5
VIN	VIN	15 16	D50	PB3	D23	15 16	D10	PD14
				PA4	D24	17 18	D9	PD15
				PB4	D25	19 20	D8	PG9
				VDDA	AVDD	1 2	D7	PG12
				AGND	AGND	3 4	D6	PA8
PA3	A0	3 4	D51	GND	GND	5 6	D5	PE11
PC0	A1	5 6	D52	PF6	A6	7 8	D4	PE14
PC3	A2	7 8	D53	PF10	A7	9 10	D3	PE13
PB1	A3	9 10	D54	PA2	A8	11 12	D2	PG14
PC2	A4	9 10	D55	PG6	D26	13 14	D1	PB6
PF11	A5	11 12	GND	PB2	D27	15 16	D0	PB7
PB2	D72	13 14	D56	GND	GND	17 18	D42	PE8
PE9	D71	15 16	D57	PD13	D28	19 20	D41	PE7
PB5	D70	17 18	D58	PD12	D29	21 22	GND	GND
PF14	D69	19 20	D59	PD11	D30	23 24	D40	PE10
PF15	D68	21 22	D60	PE2	D31	25 26	D39	PE12
GND	GND	23 24	D61	GND	GND	27 28	D38	PE6
PD0	D67	25 26	D62	PA0	D32	29 30	D37	PE15
PD1	D66	27 28	D63	PB0	D33	31 32	D36	PB10
PB14	D65	29 30	D64	PE0	D34	33 34	D35	PB11
CN9				CN10				

Figure 1.2: Presentazione dei pin presenti sul μ -controllore

1.2 Motore DC modello 28PA51G

Il motore DC modello 28PA51G³ è un motore brushless cioè senza spazzole adatto a piccole e medie potenze, questo modello in particolare è composto dal motore in sé e di un encoder integrato, ideale per raccogliere i dati in tempo reale.

³DFR ROBOT, <https://www.dfrobot.com/product-777.html>



Figure 1.3: *Motore DC modello 28PA51G*

L'encoder integrato è un encoder di tipo rotativo incrementale magnetico, modello ENC30 HEDL 5540 500IMP⁴, la sua funzione principale è quella di misurare la velocità del motore.

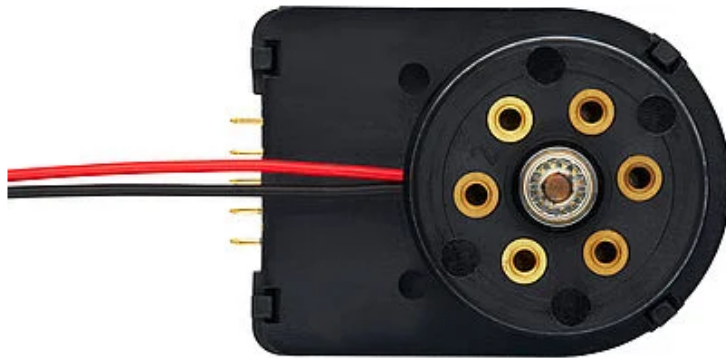


Figure 1.4: *Encoder modello ENC30 HEDL 5540 500IMP*

⁴faulhaber, <https://www.faulhaber.com/en/products/series/hedl-5540/>

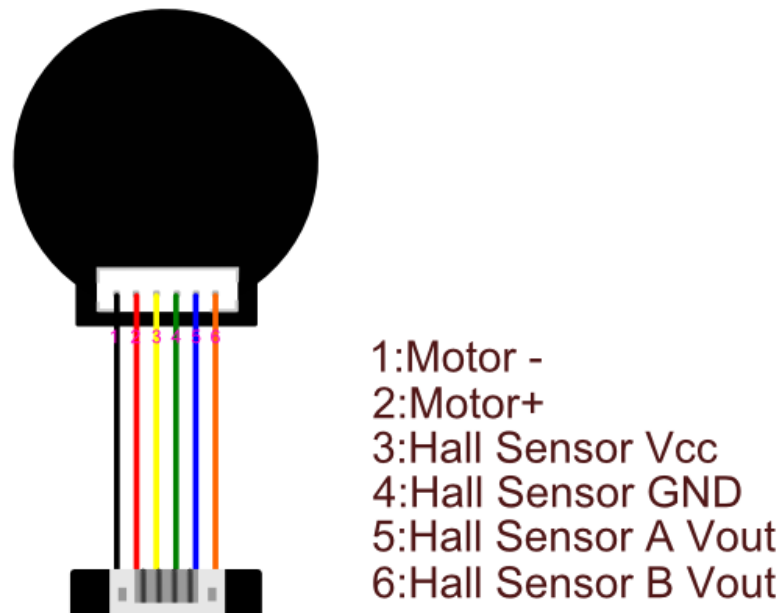


Figure 1.5: *diagramma d'uscita sul motore*

1.3 Alimentatore DC da laboratorio

L'intero sistema, in particolare il motore e i suoi componenti, necessitano di un'alimentazione elettrica di circa 12 V. Tuttavia, la scheda STM32, collegata al PC, fornisce solo 3,3 V, un valore insufficiente per alimentare direttamente il motore.

Per risolvere questa problematica, abbiamo scelto di utilizzare un generatore di tensione regolabile disponibile in laboratorio.

Di seguito, un'immagine illustrativa :



Figure 1.6: *generatore di tensione*

1.4 Encoder adapter e DC motor adapter

Al fine di semplificare la connessione tra il microcontrollore ed il motore, abbiamo usato due adattatori :

- Encoder adapter : interfaccia fornita con il motore al fine di semplificare la connessione e la lettura dei dati.

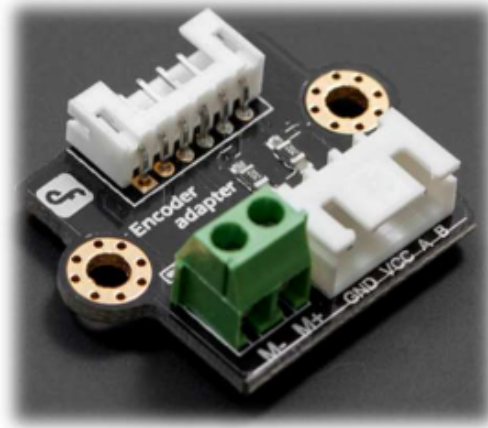


Figure 1.7: *encoder adapter*

- DC motor driver modello MD10C : questa interfaccia ci permette di gestire al meglio il funzionamento del motore; ci permette, in connessione al generatore di tensione, di fornire la tensione richiesta al motore, di gestire lo scambio di polarità che regola il senso di rotazione dell'albero e di transitare il segnale PWM mandato dalla scheda.



Figure 1.8: *DC motor driver*

1.5 Schema dei collegamenti

All'inizio, sulla scheda STM, scegliamo il Pin collegato ad un timer che possa fornire un segnale PWM, scegliendo tra un Advanced-Control TIMERS o un General Purpose TIMERS.

Abbiamo optato per il TIMER 1 collegato fisicamente al pin PE9 che sarà legato all'uscita PWM del DC MOTOR DRIVER.

Il pin PA4 configurato in GPIO output per gestire in risposta al toggler (user button) lo scambio di direzione.

Per la raccolta dei dati dall'encoder, attiviamo il TIMER4 con la modalità "encoder mode", sarà attivato automaticamente i pin PD12 e PD13 che collegheremo rispettivamente alle uscite A e B dell'Encoder Adapter.

Infine, i collegamenti di alimentazione saranno così configurati:

VCC → 3.3V della scheda STM32

GND → GND della scheda STM32

POWER + e - del DC Motor Driver → collegati al generatore di tensione.

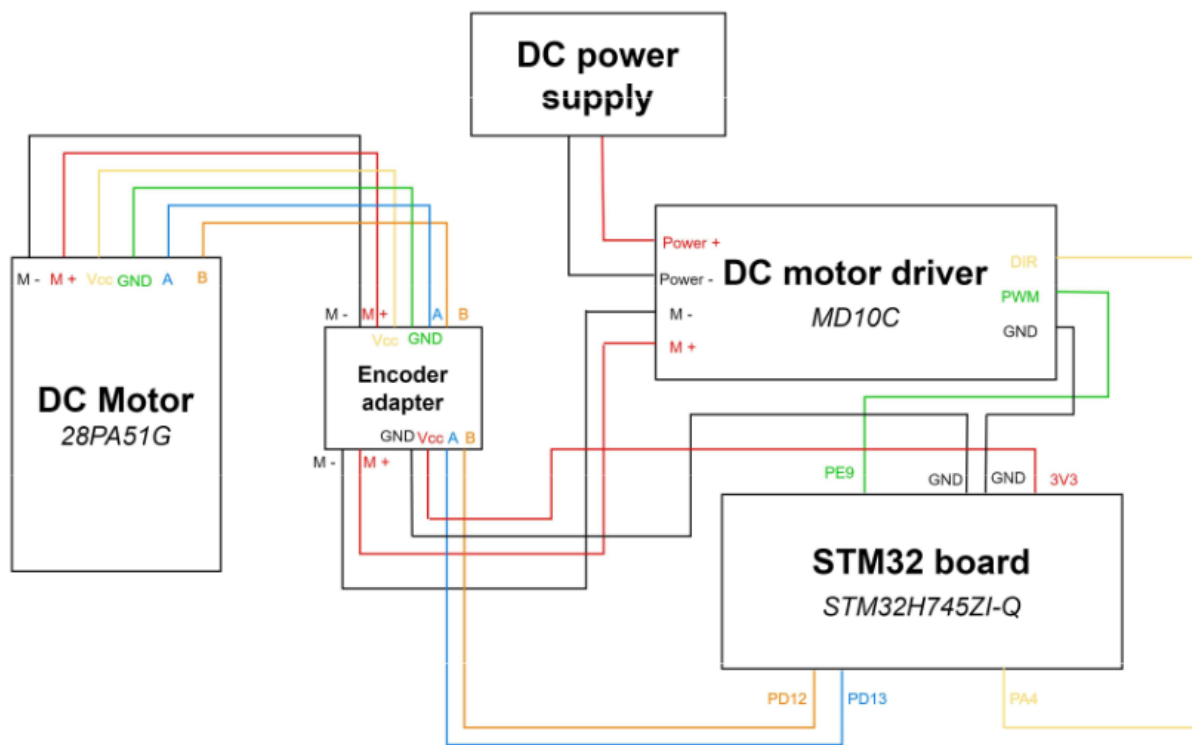


Figure 1.9: Schema dei collegamenti

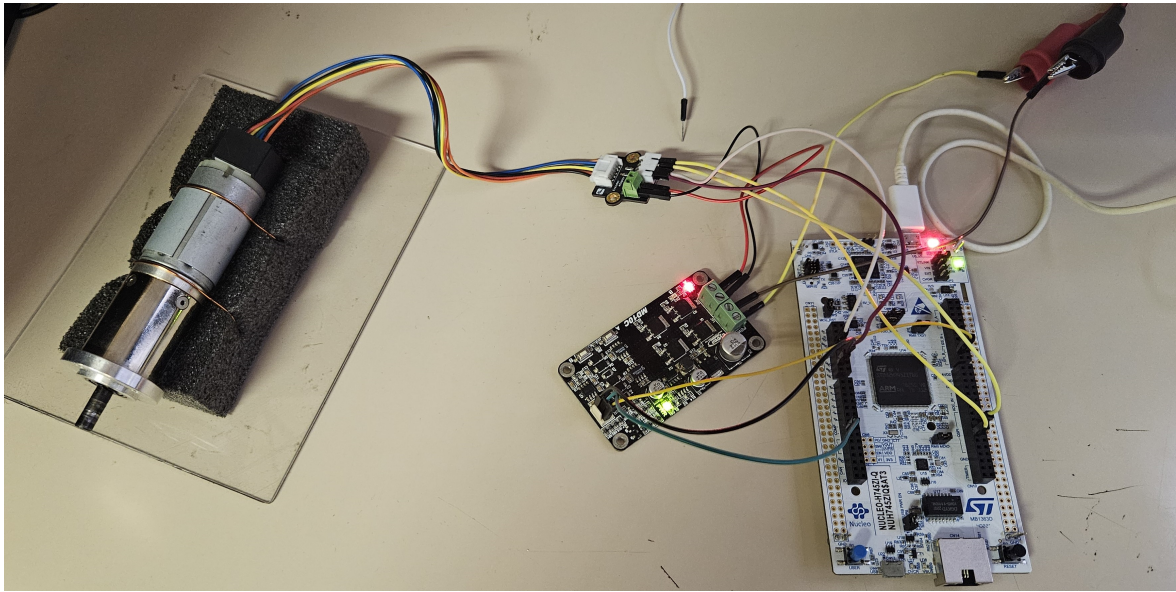


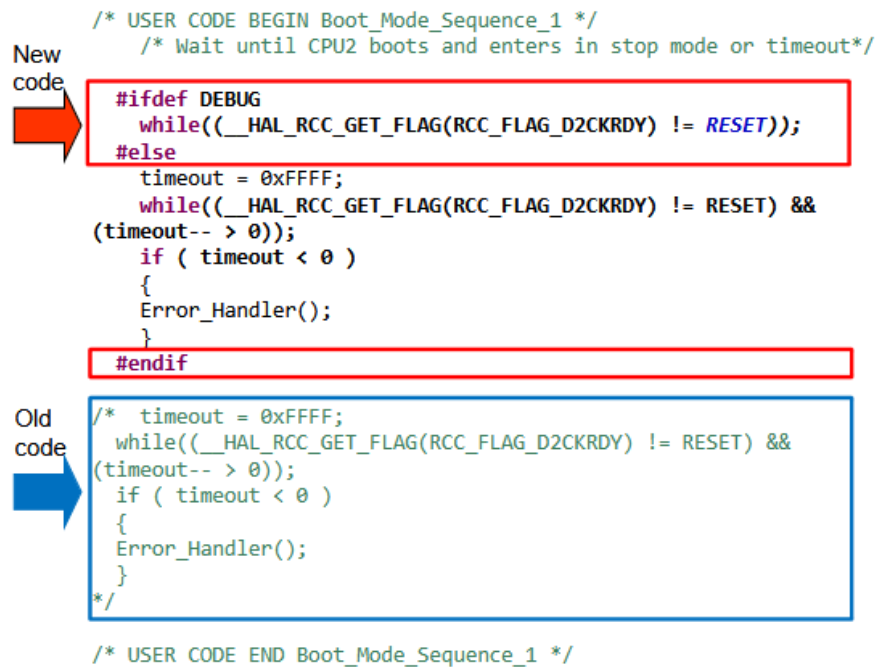
Figure 1.10: *Immagine illustrativa del sistema reale*

2 Software

Per lo svolgimento del laboratorio e lo sviluppo del codice, abbiamo utilizzato l'ambiente di sviluppo STM32CubeIDE. Inizialmente, abbiamo iniziato con la versione **1.16**, ma a causa dei numerosi problemi riscontrati durante l'implementazione, abbiamo deciso di passare alla versione **1.18**, che si è rivelata più stabile e adatta alle nostre esigenze, per la STM32CubeMX siamo sulla versione **6.14.0-RC8**.

Per visualizzare i dati del sensore in tempo reale, abbiamo utilizzato **PuTTY**, collegato al flusso di dati trasmesso tramite la USART3, che abbiamo opportunamente configurato.

La scheda STM32H745ZI-Q è dotata di due core, il Cortex-M4 e il più potente Cortex-M7; per questo progetto è stato utilizzato il core Cortex-M4. Per forzare l'avvio del programma con priorità sul core M4, è necessario inserire il seguente frammento di codice nel main del core M7.



```
/* USER CODE BEGIN Boot_Mode_Sequence_1 */
/* Wait until CPU2 boots and enters in stop mode or timeout*/

New code
#ifdef DEBUG
while((__HAL_RCC_GET_FLAG(RCC_FLAG_D2CKRDY) != RESET));
#else
timeout = 0xFFFF;
while((__HAL_RCC_GET_FLAG(RCC_FLAG_D2CKRDY) != RESET) &&
(timeout-- > 0));
if ( timeout < 0 )
{
Error_Handler();
}
#endif

Old code
/* timeout = 0xFFFF;
while((__HAL_RCC_GET_FLAG(RCC_FLAG_D2CKRDY) != RESET) &&
(timeout-- > 0));
if ( timeout < 0 )
{
Error_Handler();
}
*/

/* USER CODE END Boot_Mode_Sequence_1 */
```

Figure 2.1: Codice da inserire nella Main CM7

2.1 Diagramma di flusso

Ho impostato il codice in modo che il TIMER 6 generi un interrupt ogni 10 ms, consentendo l'aggiornamento delle variabili globali che rappresentano i dati dell'encoder in modo indipendente dal loop principale. Il loop principale gestisce il lampeggio del LED (200 ms) e

la visualizzazione dei dati del motore su PuTTY. Questa configurazione asincrona garantisce un aggiornamento affidabile dei dati senza essere influenzata dal tempo di stampa. In Figura 2.2 viene mostrato il diagramma esplicativo che descrive il funzionamento del sistema.

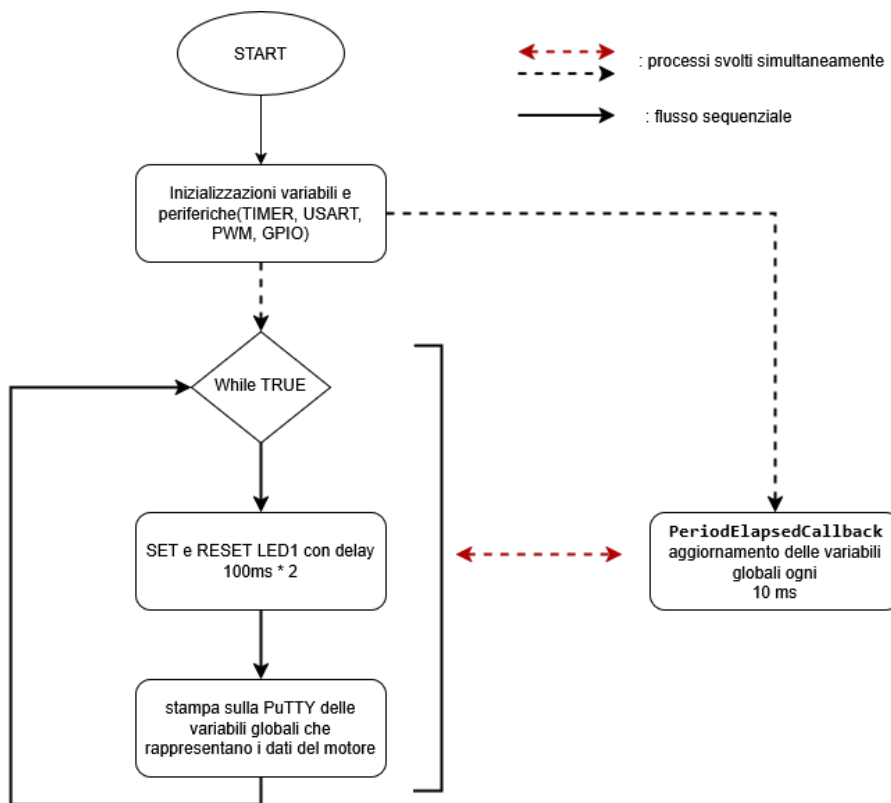


Figure 2.2: *Diagramma di flusso del codice sviluppato*

2.2 Gestione singoli componenti

In questo capitolo verrà illustrata nel dettaglio l'effettiva implementazione e configurazione delle diverse periferiche utilizzate nel nostro progetto, all'interno dell'ambiente di sviluppo STM32cubeIDE e STM32cubeMX. Si tralascerà la spiegazione relativa al lampeggio del LED, in quanto di secondaria importanza ai fini del progetto.

2.2.1 Motore

Per generare il segnale PWM, è stato attivato l'Advanced-Control Timers TIM1 in PWM mode nel file .IOC (con 240-1 come prescaler e 1000-1 come counter period). Successivamente, nel main, è stata chiamata la funzione HAL TIM PWM Start per inizializzare il flusso di generazione del segnale.

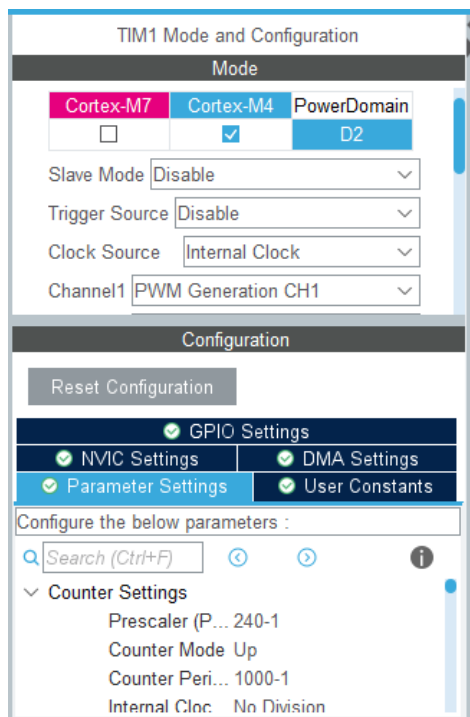


Figure 2.3: Impostazione .IOC del TIM1

poi il codice nel main :

```
/* USER CODE BEGIN 2 */

HAL_TIM_PWM_Start(&htim1,TIM_CHANNEL1);

TIM1->CCR1 = 300;
int i= TIM1->CCR1;

/* USER CODE END 2 */
```

NOTA: Bisogna notare bene che Il duty cycle predefinito è impostato a 250; tuttavia, la riga di codice "TIM1 → CCR1 = 300" risulta fondamentale perché consente di modificare dinamicamente il valore del duty cycle, ad esempio per eseguire prove e test sul sistema..

2.2.2 Encoder

Per quanto riguarda l'encoder ho configurato in questo modo :

- Attivazione del TIMER 4 in **encoder mode**.

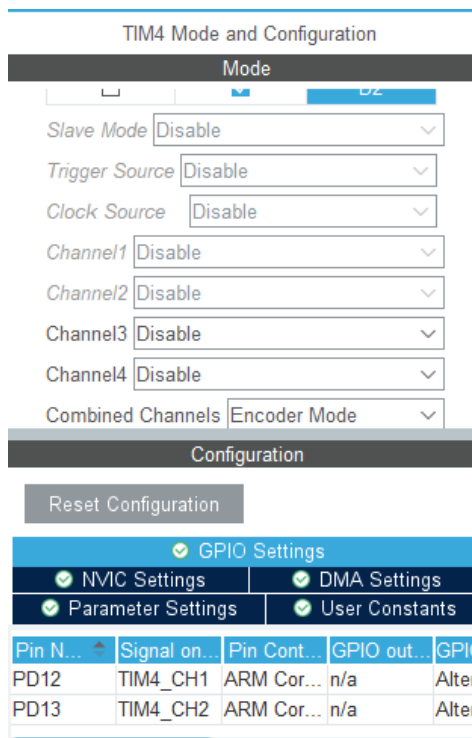


Figure 2.4: Impostazione .IOC del TIM4

- Poi inizializzazione del timer chiamando la funzione `HAL_TIM_ENCODER_START`

```
/* USER CODE BEGIN 2 */
```

```
HAL_TIM_Encoder_Start(&htim4 , TIM_CHANNEL_4);
```

```
/* USER CODE END 2 */
```

- Per la raccolta dei dati leggiamo il contatore **CNT** e calcoliamo l'angolo e la velocità equivalente.

```

/* counts è il valore dell'encoder, che varia in base alla velocità del motore
 * 2652 è il valore massimo ottenuto con il calcolo
 * counter period = ppr x gear ratio x encoder resolution
 * dunque il gioco sarà di trovare in base a count l'angolo equivalente*/

counts = TIM4 -> CNT;
delta_angle = (double) counts * 360 / 2652;

// calcolo della differenza d'angolo
diff_angle = delta_angle - old_delta_angle;

/*Correzione della differenza d'angolo in funzione della direzione
 * ---> Stessa direzione: nessuna correzione necessaria*/
if (dir == old_dir) {

    //Cambio di direzione: correzione per evitare salti
} else if (dir == 1 && diff_angle > 0) {

    diff_angle -= 360.0;

    //Cambio di direzione: correzione per evitare salti
} else if (dir == 0 && diff_angle < 0) {

    diff_angle += 360.0;

}

// Calcolo della velocità in gradi al secondo
speed_degsec = diff_angle / dt;

// Conversione diretta in rotazione per minuto (RPM)
speed = speed_degsec / 6.0;

// Aggiornamento dei valori precedenti.
old_delta_angle = delta_angle;
old_dir = dir;

```

Figure 2.5: lettura e conversione dei dati dall'encoder

la spiegazione dettagliata del codice si farà nella sezione 2.3

- Due adattatori: un encoder adapter specifico per il motore e un DC motor adapter, modello Cytron MD10C

2.2.3 Protocollo di comunicazione USART : USART3

Per stampare i dati si è pensato di sfruttare il protocollo di comunicazione USART. Per la comunicazione tramite cavo USB, nella scheda di sviluppo in questione, è stata abilitata la periferica USART3 lasciando i parametri di default:

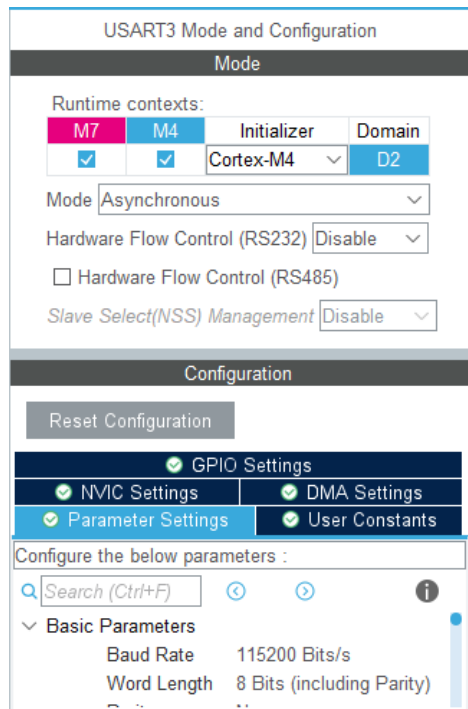


Figure 2.6: Impostazione .IOC del USART3

Di seguito attiviamo il supporto ai floating point nei parametri (non è attivato per default).

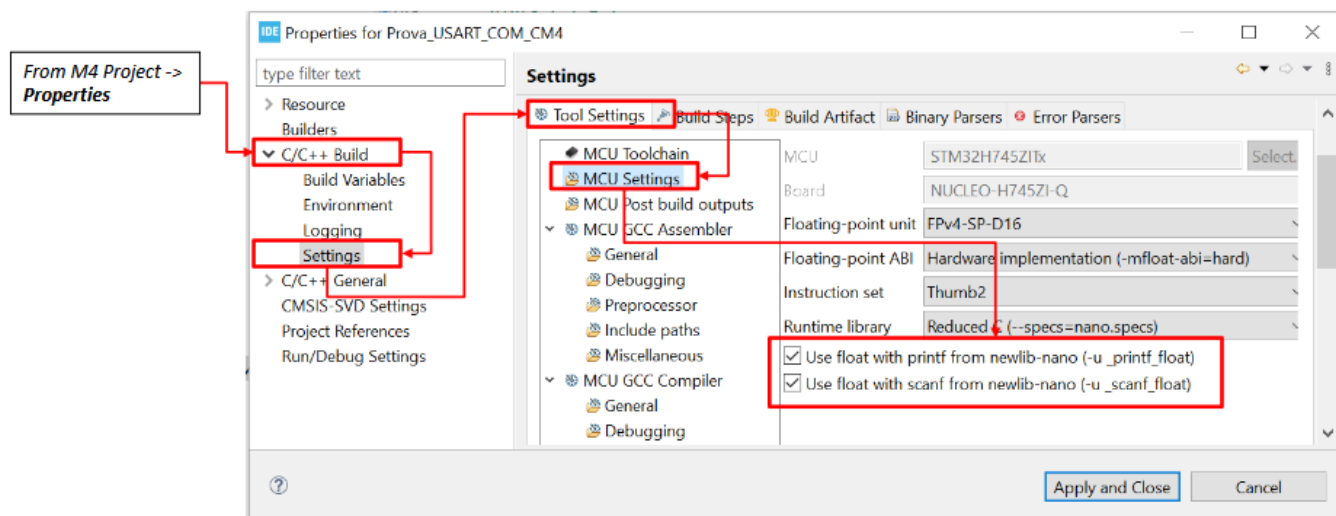


Figure 2.7: Supporto ai floating point

Dopodiché, viene inserita la funzione `_io_putchar`, la quale consente di reindirizzare l'output della printf verso il flusso di stampa della periferica USART.

```

/* USER CODE BEGIN 4 */

int __io_putchar(int ch) {
    HAL_UART_Transmit(&huart3, (uint8_t *)&ch, 1, HAL_MAX_DELAY);
    return ch;
}

finalmente con printf mandiamo i dati sul flusso USART, che sarà visualizzato sul terminale PuTTY, il printf si trova dentro il main nel ciclo While(1).

/* USER CODE BEGIN WHILE */
while (1)
{
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */

    /* External Led Toggle */
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_SET);
    HAL_Delay(100);
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_RESET);
    HAL_Delay(100);

    printf("Angolo : %.2f° | Velocità : %.2f RPM | Duty cycle : %.2f%%\r\n", delta_angle, speed, (i / 999.0) * 100);

}

/* USER CODE END 3 */

```

Figure 2.8: *printf*

2.3 Funzionamento complessivo

Come precedentemente accennato, il programma avvia due processi asincroni in esecuzione simultanea (Figura 2.2): il processo principale si occupa della stampa dei dati (Figura 2.8), mentre il secondo processo presente in figura (2.5) legge e converte il valore del contatore dell'encoder, aggiornando le variabili globali opportunamente dichiarate.

Poiché il calcolo della velocità richiede un intervallo di tempo definito ($dt = 0.01$ s), la lettura del contatore è stata vincolata a una callback gestita dal TIM6, configurato per generare un interrupt ogni 0.01 s. Questo approccio è stato scelto per garantire la massima precisione nella lettura e nel calcolo, minimizzando gli errori dovuti ai tempi di esecuzione di altri comandi. A questo livello, anche un solo millisecondo può influire sensibilmente sull'accuratezza del risultato.

```

void HAL_TIM_PeriodElapsedCallback (TIM_HandleTypeDef *htim)
{
    // la callback è stato configurato per generare l'interrupt ogni 10ms
    if (htim == &htim6)
    {
        dt = 0.01;

        /* counts è il valore dell'encoder, che varia in base alla velocità del motore
         * 2652 è il valore massimo ottenuto con il calcolo
         * counter period = ppr x gear ratio x encoder resolution
         * dunque il gioco sarà di trovare in base a count l'angolo equivalente*/

        counts = TIM4 -> CNT;
        delta_angle = (double) counts *360/ 2652;
    }
}

```

Figure 2.9: *funzione di callback, in realtà è la stessa funzione presente sulla figura 2.5, cioè il calcolo si fa dentro la funzione della callback*

2.3.1 Spiegazione del calcolo effettivo

la figura 2.5 mostra il codice responsabile della posizione angolare del motore, ricavata dal registro di conteggio del timer TIM4, e del calcolo della velocità angolare istantanea in giri al minuto (RPM). Il procedimento considera inoltre la direzione di rotazione e gestisce eventuali discontinuità associate ai cambi di direzione.

```

counts = TIM4 -> CNT;
delta_angle = (double) counts *360/ 2652;

```

Il valore ottenuto dall'encoder (nella variabile **counts**) viene convertito in gradi mediante un rapporto di proporzionalità, che tiene conto della risoluzione complessiva del sistema di misura, pari in questo caso a 2652 impulsi per giro. Tale valore è stato calcolato applicando la seguente formula, fornita dal costruttore del motore:

$$\text{Counter Period} = \text{PPR} \times \text{Gear Ratio} \times \text{Encoder Resolution} \quad (2.1)$$

dove:

- **PPR** è il numero di impulsi per rivoluzione generati dall'encoder;
- **Gear Ratio** rappresenta il rapporto di riduzione dell'ingranaggio;
- **Encoder Resolution** è il numero di incrementi generati per ogni impulso base.

```
diff_angle = delta_angle - old_delta_angle;
```

Successivamente, si calcola la differenza angolare tra la posizione corrente e quella precedentemente registrata (`old_delta_angle`). Questo valore rappresenta l'incremento (o decremento) angolare avvenuto in un intervallo di tempo prefissato `dt`.

```
if (dir == old_dir) {  
    // nessuna correzione necessaria  
} else if (dir == 1 && diff_angle > 0) {  
    diff_angle -= 360.0;  
} else if (dir == 0 && diff_angle < 0) {  
    diff_angle += 360.0;  
}
```

In questa parte viene effettuata una correzione condizionale della `diff_angle` nel caso in cui venga rilevato un cambio di direzione del motore (cioè `dir` diverso da `old_dir`). La correzione serve ad evitare salti improvvisi nel valore angolare, ad esempio durante il passaggio da 359° a 0° o viceversa, che altrimenti verrebbero interpretati erroneamente come una grande variazione di velocità.

```
speed_degsec = diff_angle / dt;  
speed = speed_degsec / 6.0;
```

La velocità angolare viene calcolata dividendo `diff_angle` per l'intervallo di campionamento `dt`, ottenendo così un valore in gradi al secondo. Successivamente, si effettua la conversione in RPM dividendo per 6, poiché: $1 \text{ RPM} = 360^\circ / 60 \text{ s} = 6^\circ/\text{s}$

```
old_delta_angle = delta_angle;  
old_dir = dir;
```

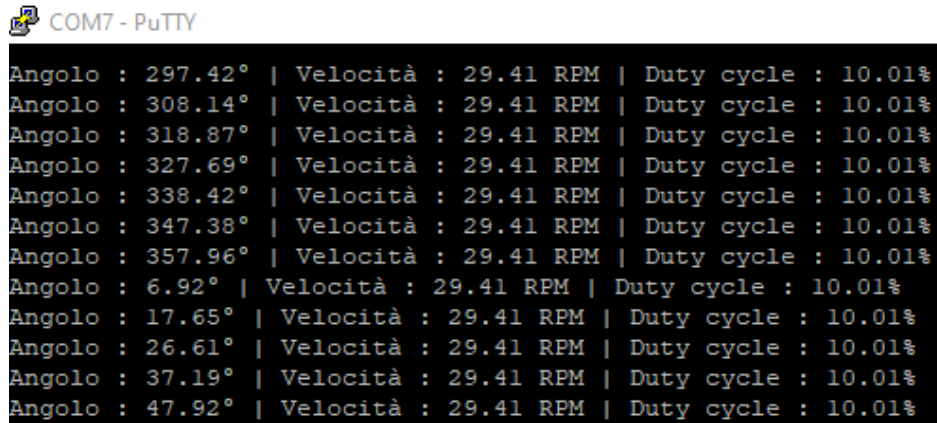
Infine, si aggiornano le variabili storiche per permettere un corretto calcolo al prossimo ciclo di interrupt.

3 Test e risultati

I test effettuati nell'ambito di questo progetto si sono concentrati principalmente sulla verifica dell'accuratezza dei dati acquisiti dall'encoder, in corrispondenza ai valori del duty cycle applicati al motore, ovvero con la percentuale di potenza erogata.

A tal fine, i dati rilevati sono stati visualizzati tramite l'interfaccia seriale su PuTTY, in corrispondenza dei diversi valori di duty cycle impostati. Questo approccio ha consentito, durante lo sviluppo, di monitorare in tempo reale il comportamento del sistema, facilitando l'ottimizzazione delle elaborazioni e la valutazione dell'affidabilità delle misure.

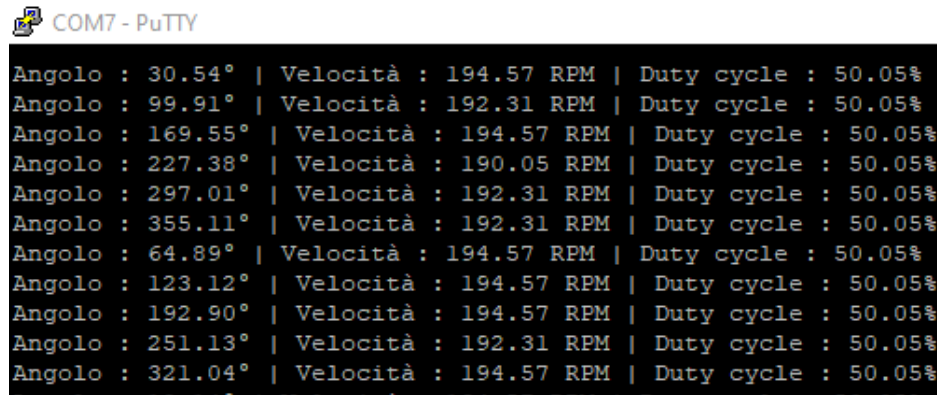
Qui sotto ci sono i dati visualizzati nella finestra di PuTTY.



COM7 - PuTTY

Angolo	Velocità	Duty cycle
297.42°	29.41 RPM	10.01%
308.14°	29.41 RPM	10.01%
318.87°	29.41 RPM	10.01%
327.69°	29.41 RPM	10.01%
338.42°	29.41 RPM	10.01%
347.38°	29.41 RPM	10.01%
357.96°	29.41 RPM	10.01%
6.92°	29.41 RPM	10.01%
17.65°	29.41 RPM	10.01%
26.61°	29.41 RPM	10.01%
37.19°	29.41 RPM	10.01%
47.92°	29.41 RPM	10.01%

Figure 3.1: *duty cycle 10%*



COM7 - PuTTY

Angolo	Velocità	Duty cycle
30.54°	194.57 RPM	50.05%
99.91°	192.31 RPM	50.05%
169.55°	194.57 RPM	50.05%
227.38°	190.05 RPM	50.05%
297.01°	192.31 RPM	50.05%
355.11°	192.31 RPM	50.05%
64.89°	194.57 RPM	50.05%
123.12°	194.57 RPM	50.05%
192.90°	194.57 RPM	50.05%
251.13°	192.31 RPM	50.05%
321.04°	194.57 RPM	50.05%

Figure 3.2: *duty cycle 50%*

Ho realizzato i test con duty cycle 10%, 30%, 50%, 70%, 90%; e ho fatto un grafico di corrispondenza per vedere l'andamento sulla piattaforma online Google Colab ¹.

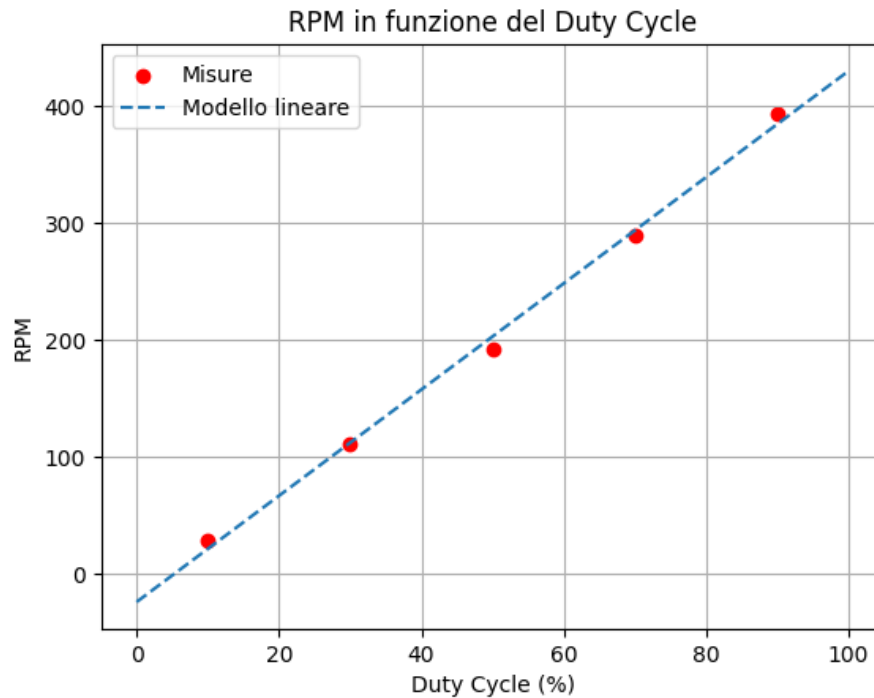


Figure 3.3: *grafico duty cycle - RPM*

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

# Dati
duty = np.array([10, 30, 50, 70, 90]).reshape(-1, 1)
rpm = np.array([29.41, 110.86, 192.31, 289.59, 393.93])

# Regressione lineare
model = LinearRegression().fit(duty, rpm)
x = np.linspace(0, 100, 200).reshape(-1, 1)
y = model.predict(x)

plt.scatter(duty, rpm, color='red', label='Misure')
plt.plot(x, y, '--', label='Modello lineare')
plt.xlabel('Duty Cycle (%)')
plt.ylabel('RPM')
plt.legend()
plt.grid()
plt.title('RPM in funzione del Duty Cycle')
plt.show()
```

Figure 3.4: *Codice usato su google colab per avere il grafico*

¹Google colab, <https://colab.research.google.com>

4 Conclusioni e sviluppi futuri

In sintesi, il progetto si focalizza sul controllo del motore attraverso un micro controllore, affrontando la programmazione a basso livello : il cuore stesso dell'automazione.

Queste competenze offrono prospettive concrete di sviluppo e permettono una comprensione più approfondita dei flussi di comunicazione che incontriamo quotidianamente. Ci aprono la strada verso applicazioni reali come: il controllo remoto di una barriera tramite telecomando, l'attivazione automatica di una porta mediante un sensore PIR (sensore di presenza), oppure l'avviamento e il controllo di un drone (un progetto già presente nel nostro laboratorio).

Il progetto, inizialmente complesso, è stato affrontato e risolto progressivamente grazie a un'attenta attività di ricerca e alle numerose ore di sperimentazione svolte in laboratorio.

Bibliografia

- [1] “STM32H745zi-q User manuel, url = https://www.st.com/resource/en/user_manual/um2408-stm32h7-nucleo144-boards-mb1363-stmicroelectronics.pdf.”
- [2] “STM32H745zi-q DATASHEET, url = <https://www.st.com/en/evaluation-tools/nucleo-h745zi-q.html>.”
- [3] “DFR ROBOT, url = <https://www.dfrobot.com/product-777.html>.”
- [4] “ENCODER, url = faulhaber, <https://www.faulhaber.com/en/products/series/hedl-5540/>.”
- [5] “MD10C DCdriver, url = <https://www.cytron.io/p-10amp-5v-30v-dc-motor-driver>.”

[1] [2] [3] [4] [5]